

Taller 4: Estándares del proyecto

EstudiaPro: plataforma de organización académica

Nicole Tatiana Acosta Tellez

Ingeniería de Sistemas, Universidad Santo Tomás, Seccional Villavicencio

Gerencia de Software

8 de octubre de 2026

1. Propósito

EstudiaPro lo desarrolla una sola persona, así que no hay un compañero que revise cada cambio. Los acuerdos de este documento se apoyan en tres cosas que no dependen de la memoria: las pruebas automáticas, una lista de revisión que se aplica a cada pull request y la validación con la interlocutora para todo lo que se ve en pantalla. Este documento reproduce el archivo ESTANDARES.md del repositorio (<https://github.com/nicoleacosta2509-bot/EstudiaPro-USTA/blob/main/ESTANDARES.md>), que se cambia por pull request como cualquier otro archivo.

2. Estilo y nombres

Tabla 1

Guías, lenguajes y herramientas

Parte	Lenguaje y herramientas	Guía y análisis estático
Backend	Java 21, Spring Boot 3.3, Maven	Google Java Style Guide (Google, s. f.): 2 espacios, 100 columnas. Sin análisis estático todavía (issue #14).
Frontend	JavaScript, React 18, Vite 5	Componentes en funciones con hooks. Análisis estático con ESLint 9 (npm run lint).

Idioma. El dominio va en español, igual que el acta (Tarea, Examen, Horario, Asignatura, Recordatorio); los términos técnicos del framework se dejan como son. Mensajes de error, comentarios y commits van en español, con tildes. Lo que ve el usuario dice qué corregir (RC-01).

Tres reglas propias de nombres. (1) Las entidades van en singular y las rutas REST en plural y minúscula (/api/tareas). (2) Una prueba se llama como una frase que dice qué pasa (rechazaDatosIncompletosYFechasInvalidas). (3) Las variables de entorno van en mayúsculas (JWT_SECRET, DB_URL) y en el repositorio solo viven valores de ejemplo.

3. Commits y ramas

Los mensajes siguen Conventional Commits (Conventional Commits, s. f.) con el formato «tipo(alcance): qué se hace, en infinitivo y en minúscula», con una primera línea de 72 caracteres como máximo y sin punto final. Ejemplo real del repositorio: «test(rendimiento): agregar el script de medición de RC-02» (commit 3f0ec1a).

Tabla 2

Tipos de commit permitidos

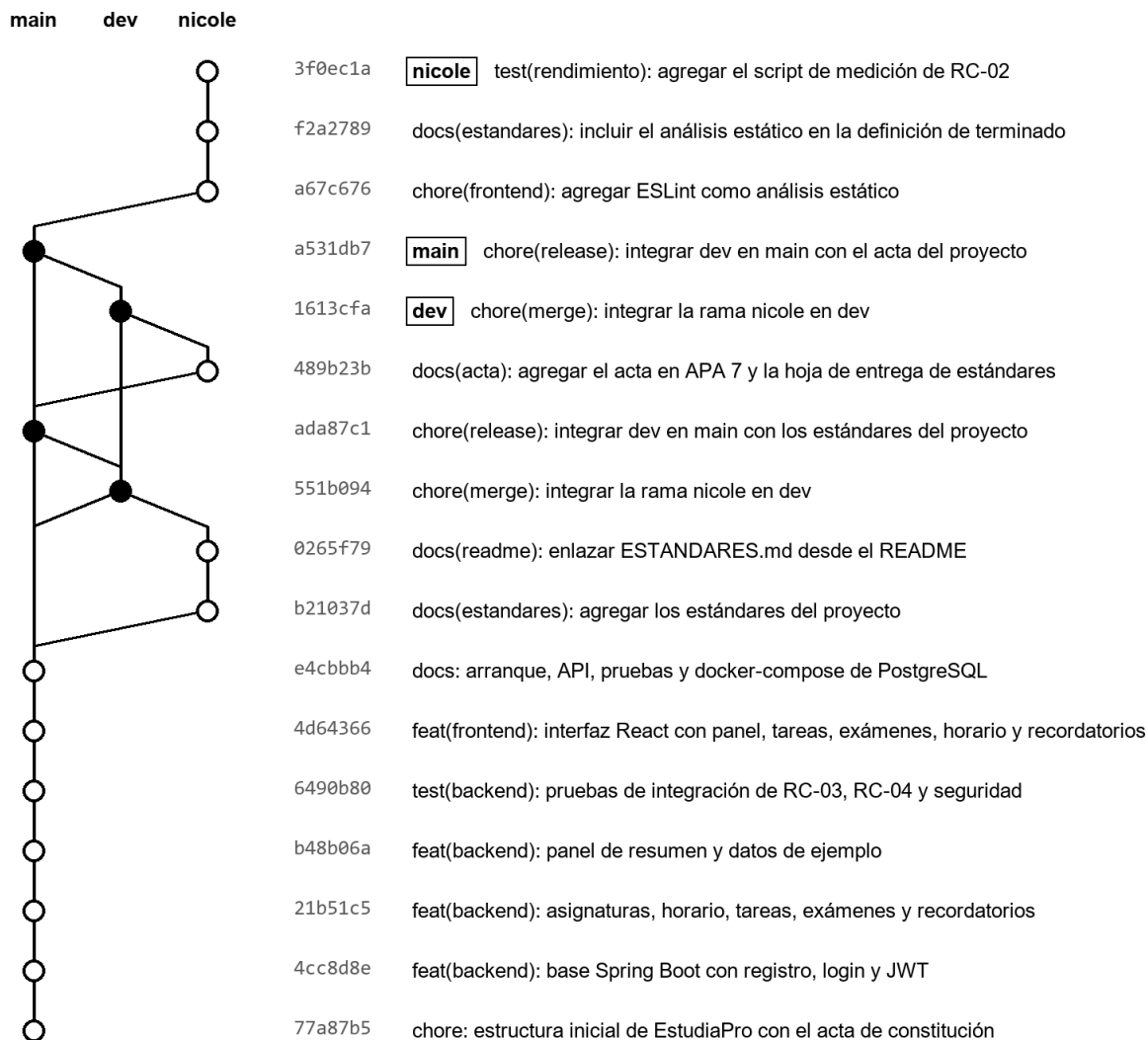
Tipo	Cuándo
feat	Funcionalidad nueva
fix	Corrección de un error
docs	Solo documentación
style	Formato, sin cambiar comportamiento
refactor	Reorganizar código sin cambiar comportamiento
test	Pruebas
chore	Configuración, dependencias e integración de ramas

Tabla 3

Ramas del repositorio

Rama	Para qué	Se integra en
main	Lo estable y entregable; solo recibe dev.	—
dev	Donde se junta lo que ya pasó la lista de revisión.	main, al cerrar el sprint
nicole	El trabajo diario.	dev, mediante pull request

Nota. Los commits de integración usan chore(merge) y chore(release). El historial de main y dev no se reescribe.

Figura 1*Historial de ramas del repositorio*

Nota. Elaboración propia a partir de `git log --graph` de github.com/nicoleacosta2509-bot/EstudiaPro-USTA, 8 de octubre de 2026. Los círculos rellenos son commits de integración.

4. Definición de listo (DoR)

Tabla 4

Criterios de la definición de listo

#	Condición
1	Existe como issue en GitHub, escrita como «como estudiante quiero [acción] para [beneficio]».
2	Tiene criterios de aceptación escritos como casos: «entrada → resultado esperado».
3	Se puede terminar en menos de 4 horas de trabajo. Si no, se parte.
4	Está dentro del alcance del acta; si es una función nueva, ya se evaluó contra las exclusiones.
5	Si toca backend y frontend, la ruta, el método, el cuerpo y los códigos están escritos en el issue.
6	Si cambia algo que usa la interlocutora, se sabe cuándo se le va a mostrar.

5. Definición de terminado (DoD)

La definición de terminado es el compromiso de calidad de cada incremento (Schwaber y Sutherland, 2020). Una historia está terminada cuando cumple las condiciones de la Tabla 5.

Tabla 5

Criterios de la definición de terminado

#	Condición	Cómo se comprueba
1	Las pruebas del backend pasan.	cd backend && mvn test termina sin fallos.
2	Cada criterio de aceptación de la API tiene una prueba automática.	Hay una prueba en backend/src/test cuyo nombre describe el criterio, y pasa.
3	El frontend pasa el análisis estático y compila.	npm run lint y npm run build terminan con código 0.
4	Si el cambio es de interfaz, el pull request trae captura y pasos.	El pull request incluye la imagen y los pasos con npm run dev.
5	Los formularios validan y muestran el error junto al campo.	Se envía vacío y con fecha pasada; cada campo muestra su mensaje.
6	Si cambia un endpoint, una variable o un comando, cambia el README.	El diff incluye el README correspondiente.
7	El pull request pasó la lista de revisión.	La lista aparece marcada en la descripción.
8	Si cambia algo que usa la interlocutora, ella lo vio.	Su observación queda en el issue o en el acta de validación.

6. Política de revisión

Al ser una sola persona, la revisión se hace al día siguiente de terminar el cambio, con esta lista, que se copia en cada pull request (ejemplo:

<https://github.com/nicoleacosta2509-bot/EstudiaPro-USTA/pull/16>):

1. mvn test, npm run lint y npm run build pasan.
2. No hay contraseñas, llaves, tokens ni archivos .env en el diff.
3. Toda consulta nueva filtra por el estudiante autenticado.
4. Lo nuevo en el backend tiene una prueba que lo cubre.
5. Los mensajes de error dicen qué corregir.
6. El README está al día si cambió un endpoint, una variable o un comando.
7. Los mensajes de commit siguen la convención.

Bloquea la integración cualquier casilla sin marcar, algo que antes funcionaba y dejó de funcionar, o conflictos con dev. No bloquean los nombres que igual cumplen la guía, los refactors sin cambio de comportamiento ni las optimizaciones que nadie ha medido: se anotan como mejora en un issue.

7. Cuando esto no se pueda cumplir

Tabla 6

Escenarios y qué se hace

Escenario	Qué se hace
Semana de parciales y no hay tiempo para revisar al día siguiente.	Se revisa antes de integrar en main, nunca después; lo no revisado se queda en dev.
La interlocutora no puede en la semana de la demo.	Se le envían capturas y pasos y se agenda la validación para la semana siguiente.
La noche antes de una entrega falla una prueba.	Se corrige o se deja fuera; no se entrega a medias.
No hay PostgreSQL a mano.	Las pruebas corren con H2 en memoria, así que la DoD se cumple igual.

8. Lo que todavía no se cumple

- No hay formateador automático ni análisis estático en el backend (issue #14).
- El frontend no tiene pruebas automáticas; por eso la DoD pide captura y pasos.
- Falta la prueba en Microsoft Edge (issue #12) y repetir RC-02 con PostgreSQL (issue #11).
- La versión 1.0 se hizo antes de estos acuerdos, sin issues ni pull requests.

9. Aceptación

Tabla 7

Aceptación de los estándares

Nombre completo	Aceptación	Fecha
Nicole Tatiana Acosta Tellez	Conozco y acepto estos estándares.	7 de octubre de 2026

Referencias

Conventional Commits. (s. f.). *Conventional Commits 1.0.0*.

<https://www.conventionalcommits.org/es/v1.0.0/>

Google. (s. f.). *Google Java style guide*. <https://google.github.io/styleguide/javaguide.html>

Schwaber, K., y Sutherland, J. (2020). *La guía de Scrum*.

<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-Latin-South-American.pdf>